

LAB : Prompt Engineering pour les systèmes multi agents

Création d'un nouveau projet

Créer un dossier de TP et ouvrez le dans vscode

Ouvrez un nouveau terminal et vérifiez le version de python et uv

```
PS C:\Users\user\Desktop\langchainPrompt> python --version
Python 3.12.10
PS C:\Users\user\Desktop\langchainPrompt> uv --version
uv 0.6.14 (a4cec56dc 2025-04-09)
```

Pour installer uv :

En macOS/Linux:

Utiliser curl pour télécharger le script et l'exécuter avec sh:

```
curl -Lsf https://astral.sh/uv/install.sh | sh
```

Si votre système n'a pas curl utilisez :

```
wget -qO- https://astral.sh/uv/install.sh | sh
```

En Windows:

Utiliser irm et iex

```
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

En cas de problème: <https://docs.astral.sh/uv/getting-started/installation/>

Créer un nouveau projet avec uv

```
PS C:\Users\user\Desktop\langchainPrompt> uv init
Initialized project `langchainprompt`
PS C:\Users\user\Desktop\langchainPrompt> uv venv
Using CPython 3.12.10 interpreter at: C:\Users\user\
Python312\python.exe
Creating virtual environment at: .venv
Activate with: .venv\Scripts\activate
```

Activez l'environnement virtuelle

```
PS C:\Users\user\Desktop\langchainPrompt> .venv\Scripts\activate
(langchainPrompt) PS C:\Users\user\Desktop\langchainPrompt> █
```

En cas de problème de permission :

```
.venv\Scripts\activate : Impossible de charger le fichier
C:\Users\user\Desktop\langchainPrompt\.venv\Scripts\activate.ps1, car
l'exécution de scripts est désactivée sur ce système. Pour plus d'informations,
consultez about_Execution_Policies à l'adresse
https://go.microsoft.com/fwlink/?LinkID=135170.
```

Au caractère Ligne:1 : 1

```
+ .venv\Scripts\activate
```

```
+ ~~~~~
```

```
+ CategoryInfo          : Erreur de sécurité : (:) [], PSSecurityException
```

```
+ FullyQualifiedErrorId : UnauthorizedAccess
```

1. Ouvrez PowerShell en Administrateur :

- Cliquez sur le menu Démarrer, tapez "PowerShell".
- Faites un clic droit sur "Windows PowerShell" et choisissez "Exécuter en tant qu'administrateur".

2. Exécutez la commande de déblocage :

- Tapez la commande suivante et appuyez sur Entrée

Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser

Exécutez uv add ipkernel

▼ LANGCHAINPROMPT

> .venv

≡ .python-version

🔗 main.py

📄 prompt.ipynb

⚙️ pyproject.toml

📖 README.md

≡ uv.lock

```
(langchainPrompt) PS C:\Users\user\Desktop\langchainPrompt> uv add ipykernel
Resolved 34 packages in 2.14s
Prepared 19 packages in 6.89s
Installed 28 packages in 3.15s
```

Sélectionnez un kernel

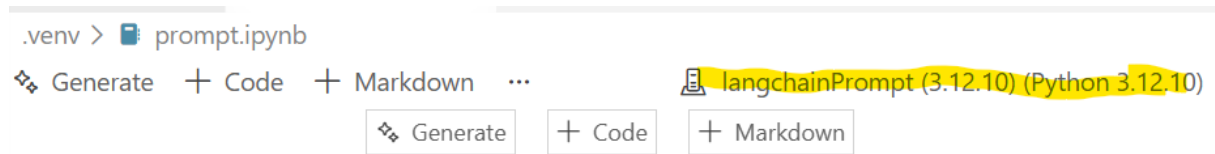
.venv > 📄 prompt.ipynb

🔗 Generate + Code + Markdown | ▶ Run All ✖ Clear All Outputs ... 📄 Select Kernel

🔗 Generate

+ Code

+ Markdown



La tokenisation avec Tiktoken

```
import tiktoken
encoding = tiktoken.encoding_for_model("gpt-4o")
print(encoding.name)

system_message = """
Perform Sentiment analysis of the review presented in the user message.
The result should be positive or negative. Do not justify your response
"""

tokens = encoding.encode(system_message)

print(len(tokens))
print(tokens)
for token in tokens:
    print(encoding.decode_single_token_bytes(token=token),end="")
def num_tokens_from_string(string: str, encoding_name: str = "o200k_base") ->
int:
    """Returns the number of tokens in a text string."""
    encoding = tiktoken.get_encoding(encoding_name)
    num_tokens = len(encoding.encode(string))
    return num_tokens

num_tokens_from_string("tiktoken is great!")
```

Encoding name 	OpenAI models
<code>o200k_base</code>	gpt-4o, gpt-4o-mini, gpt-5
<code>cl100k_base</code>	gpt-4-turbo, gpt-4, gpt-3.5-turbo, text-embedding-ada-002, text-embedding-3-small, text-embedding-3-large
<code>p50k_base</code>	Codex models, <code>text-davinci-002</code> , <code>text-davinci-003</code>
<code>r50k_base</code> (or <code>gpt2</code>)	GPT-3 models (e.g., <code>davinci</code>)

Comment formuler des prompts pour les LLM locaux avec Ollama

```
from langchain_ollama import ChatOllama

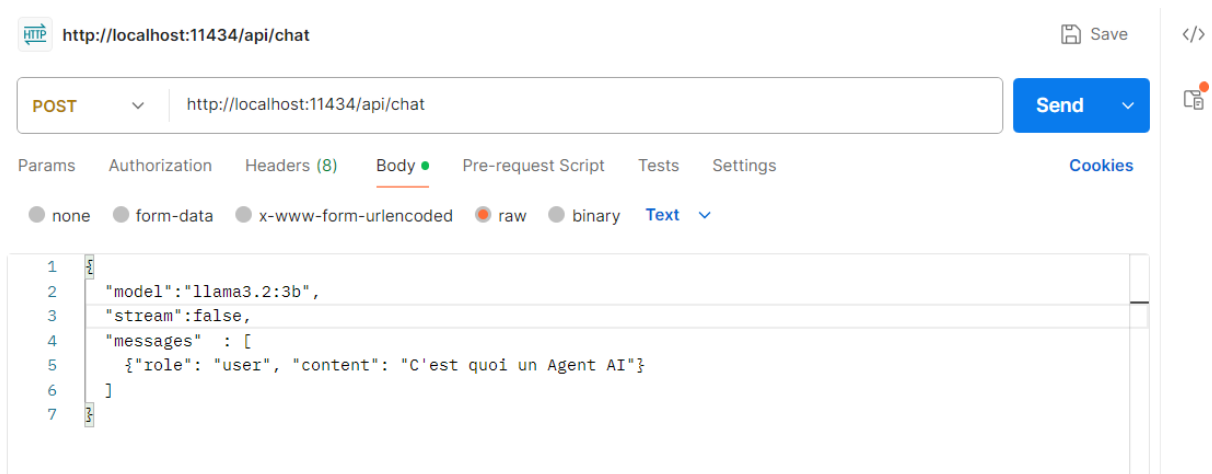
llm = ChatOllama(model="llama3.2:3b")

response = llm.invoke([
    {"role": "system", "content": "You are a helpful assistant. The output should be in Markdown"},
    {"role": "user", "content": "C'est quoi un Agent AI"}
])

from IPython.display import display, Markdown

print(display(Markdown(response.content)))
```

Interagir avec les LLM Ollama en utilisant n'importe quel client HTTP (Postman)



Comment rédiger des instructions pour les LLMs Groq

```
from dotenv.ipython import load_dotenv
load_dotenv(override=True)
```

⚙️ .env

```
1 OPENAI_API_KEY=sk-proj-wujEJzUK_v5zNBID_CDsMssqUVb9coAr3U
2 Groq_API_Key=gsk_k5fsftwCZtXCRse8bGvkWGdyb3FYfCyW8os2vhch
```

```
from langchain_groq import ChatGroq
llm2 = ChatGroq(model="openai/gpt-oss-120b")
```

```
from langchain.messages import SystemMessage, HumanMessage
resp2 = llm2.invoke([
    SystemMessage("You are a helpful assistant. The output should be in Mark-
down"),
    HumanMessage("C'est quoi un Agent AI")
])

print(display(Markdown(resp2.content)))
```

Comment rédiger des instructions pour les LLMs OpenAI

```
from langchain_openai import ChatOpenAI
load_dotenv(override=True)
```

⚙️ .env

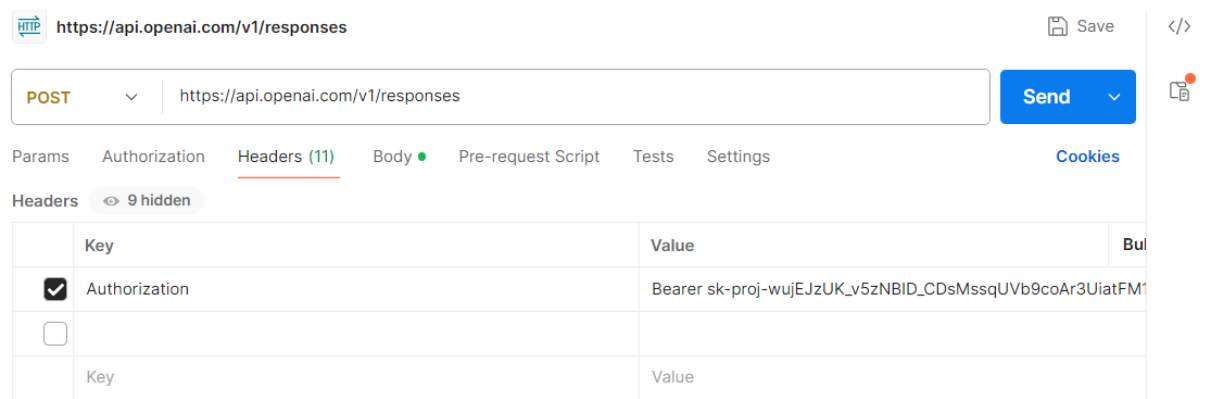
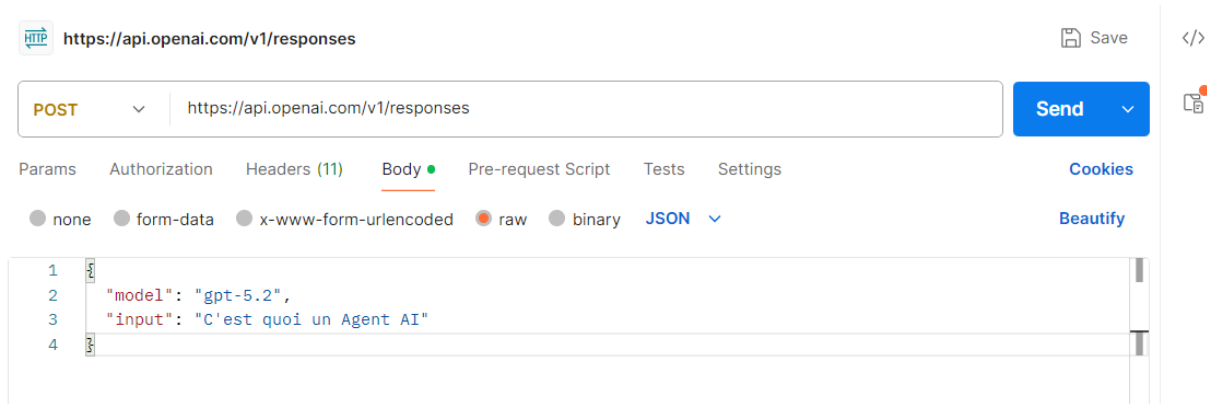
```
1 OPENAI_API_KEY=sk-proj-wujEJzUK_v5zNBID_CDsMssqUVb9coAr3U
2 Groq_API_Key=gsk_k5fsftwCZtXCRse8bGvkWGdyb3FYfCyW8os2vhch
```

```
model = ChatOpenAI(
    model="gpt-5.2", temperature=0
)

response = model.invoke([
    {"role": "system", "content": "You are a helpful assistant. The output
should be in Markdown"},
    {"role": "user", "content": "C'est quoi un Agent AI"}
])

from IPython.display import display, Markdown
print(display(Markdown(response.content)))
```

Interagir avec les LLM OpenAI en utilisant n'importe quel client HTTP (Postman)



Exemple de prompt : analyse de sentiment basée sur les aspects

system_message = """

Effectuez une analyse de sentiments basée sur les aspects des avis concernant les ordinateurs portables présentés en entrée.

Chaque avis peut comporter un ou plusieurs des aspects suivants : screen, keyboard et pad.

Pour chaque avis présenté en entrée :

- Identifiez la présence d'au moins un des trois aspects (screen, keyboard, pad).
- Attribuez une polarité de sentiment (positive, negative ou neutral) à chaque aspect. Organisez votre réponse dans un objet JSON avec les en-têtes suivants :

- category:[liste des aspects]
- polarity:[liste des polarités correspondantes pour chaque aspect]

Si l'un des aspects n'est pas présent dans l'avis de l'utilisateur, tu supposes que la polarité est neutre

"""

```
llm = ChatOpenAI(
    model="gpt-5.2",
    temperature=0,
    model_kwargs={
        "response_format": {"type": "json_object"}
    }
)

resp = llm.invoke([
    {"role": "system", "content": system_message},
    {"role": "user", "content": "L'écran est très bon, mais je n'ai pas aimé
la souris. le clavier Ma fih Maytchaf"}
])

print(resp.content)
```

```
import json
result = json.loads(resp_text_json)
type(result)
```

```
result
```

```
print(result['polarity'][0])
```

LLM Multi-Modal : génération d'image

```
llm4 = ChatOpenAI(model="gpt-5.2")
llm_with_tools = llm4.bind_tools([
    {"type": "image_generation", "quality": "high"}
])

resp4 = llm_with_tools.invoke(input=[
    HumanMessage("Je veux une photo d'un chat qui code du java")
])
```

```
from IPython.display import Image
import base64
```

```
Image(base64.b64decode(resp4.content_blocks[0]['base64']))
```

LLM Multi-Modal : Description d'image

```
path = "rag.png"
Image(path)
llm5 = ChatOpenAI(model="gpt-5.2")

def encode_image(image_path):
    with open(image_path, "rb") as image_file:
        return base64.b64encode(image_file.read()).decode("utf-8")

img = encode_image(path)

resp5 = llm5.invoke(input=[
    HumanMessage(content=[
        {"type": "text", "text": "Qu'est ce que tu vois dans cette image"},
        {"type": "image_url", "image_url": {"url":
f"data:image/png;base64,{img}"}}
    ])]

print(display(Markdown(resp5.content)))
```